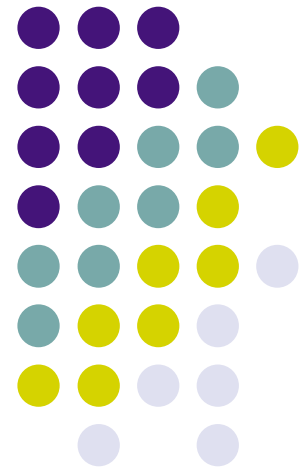


Transacciones Concurrencia

II - Semestre 2007





Transacciones

- Una **transacción** es una unidad lógica de procesamiento de la base de datos que incluye una o más operaciones de acceso a la base de datos, que pueden ser de inserción eliminación, modificación o recuperación.
- Las transacciones pueden delimitarse con sentencias explícitas **begin transaction** and **end transaction**



Transacción (cont.)

- Asumimos que una base de datos es modelada en forma simplificada como una colección de elementos de datos con nombre (registro, bloque en disco, páginas..)
- Las operaciones básicas son:
 - Leer_elemento(X)
 - Escribir_elemento(X)



Transacción (cont.)

- Leer_elemento(X):
 - Encontrar la dirección del bloque de disco que contiene el elemento X
 - Copiar ese bloque de disco en un almacenamiento intermedio (buffer) dentro de la memoria principal (si es que ese bloque no está ya en algún buffer de la memoria principal).
 - Copiar el elemento X del almacenamiento intermedio en la variable de programa X



Transacción (cont.)

- escribir_elemento(X):
 - Encontrar la dirección del bloque de disco que contiene el elemento X
 - Copiar ese bloque de disco en un almacenamiento intermedio (buffer) dentro de la memoria principal (si es que ese bloque no está ya en algún buffer de la memoria principal).
 - Copiar el elemento X desde la variable del programa X en el lugar correcto dentro del buffer.
 - Almacenar el bloque actualizado desde el buffer al disco



Concurrencia

- En sistemas multiusuario, es necesario un mecanismo para controlar la concurrencia. Se pueden producir inconsistencias importantes derivadas del acceso concurrente (3 problemas)
- Las transacciones de los usuarios se podrían ejecutar de manera concurrente y podrían acceder y actualizar los mismos elementos de la BD.



Control de Concurrencia

- Problemas:
 - Actualización perdida
 - Actualización temporal (lectura sucia)
 - Resumen incorrecto
- Sistema de BD de reservas en una línea área.
- T1: transfiere N reservas de un vuelo, cuyo número de asientos reservados está almacenado en el elemento de la BD llamado X, a otro vuelo, cuyo número de asientos reservados está almacenado en el elemento de la BD llamado Y



Actualización Perdida

- Esto ocurre cuando las transacciones que tienen acceso a los mismos elementos de la BD tienen sus operaciones intercaladas de modo que hacen incorrecto el valor de algún elemento.
- T1 y T2 se introducen al mismo tiempo y sus operaciones se intercalan.



Actualización Perdida

- T1

- Read-item(X);
- $X := X - N$;
- Write-item(X);
- Read-item(Y);
- $Y := Y + N$;
- Write-item(Y);

TIEMPO

- T2

- Read-item(X);
- $X := X + M$;
- write-item(X);

El elemento X tiene un valor incorrecto porque la actualización realizada por T1 se ha perdido (sobrescrito)



Actualización perdida

- El valor final del elemento X es incorrecto, porque T2 lee el valor de X ANTES de que T1 lo modifique en la BD, con lo que se pierde el valor actualizado que resulta de T1.
- Si $X=80$ al principio, $N=5$ (T1 transfiere 5 reservas de asientos del vuelo que corresponde a X al vuelo que corresponde a Y) y $M=4$ (T2 reserva 4 asientos en X), el resultado final debería ser $X=79$, pero es $X=84$, porque la actualización de T1 que eliminó 5 asientos de X se ha perdido.



Actualización Temporal

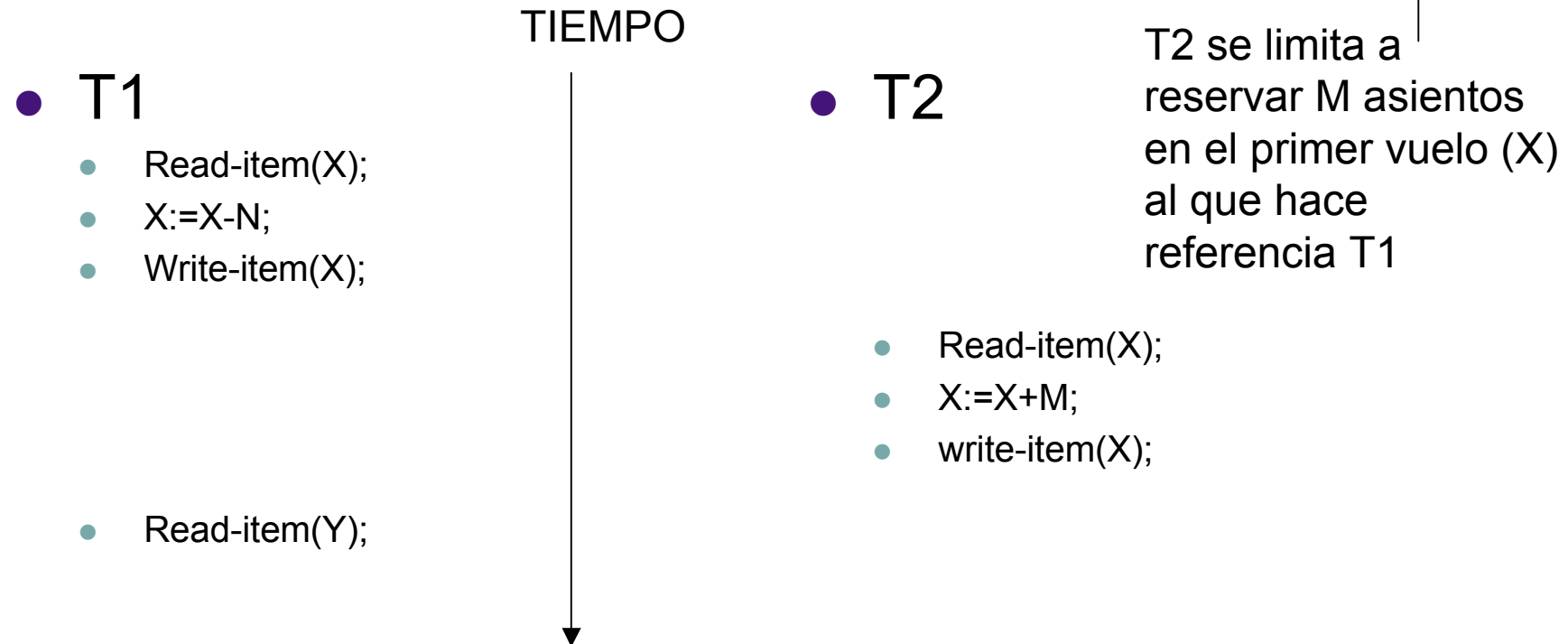
- Esto ocurre cuando una transacción actualiza un elemento de la BD y luego la transacción falla por alguna razón. Otra transacción tiene acceso al elemento actualizado antes de que se restaure a su valor original.
- T1 actualiza el elemento X y después falla antes de completarse, así que el sistema debe cambiar X otra vez a su valor original.



Actualización Temporal

- Antes de que pueda hacerlo, la transacción T2 lee el valor “temporal” de X, que no se grabará permanentemente en la BD debido al fallo de T1.
- El valor que T2 lee de X se llama dato sucio, porque fue creado por una transacción que no se ha completado ni confirmado todavía.

Actualización Temporal



La T1 falla y debe restaurar X a su antiguo valor; mientras tanto T2 ha leído el valor temporal incorrecto de X



Resumen Incorrecto

- Si una transacción está calculando una función agregada de resumen sobre varios registros mientras otras transacciones están actualizando algunos de ellos, puede ser que la función agregada calcule algunos valores antes de que se actualicen y otros después de actualizarse



Resumen Incorrecto

TIEMPO

- T1

- Read-item(X);
- $X := X - N$;
- Write-item(X);

- Read-item(Y);
- $Y := Y + N$;
- Write-item(Y);



- T3

- Sum:=0;
- Read-item(A);
- Sum:=sum+A;

- Read-item(X);
- Sum:=sum+X;
- Read-item(Y);
- Sum:=sum+Y;

T3 lee X después de restarle N y lee Y antes de sumarsele N, así que el resultado es un resumen incorrecto (discrepancia de N)

Manejador de Transacciones



- Siempre que se introduce una transacción a un SGBD para ejecutarla, el sistema tiene que asegurarse de que:
 - Todas las operaciones de la transacción se realicen con éxito y su efecto quede registrado permanentemente en la BD
 - Que la transacción no tenga efecto alguno sobre la BD ni sobre otra transacción, si es que no se completa o falla.



Tipos de Fallos

- Caída del Sistema: durante la ejecución de la transacción se produce un error de hardware, software o red.
- Error de transacción o del sistema: alguna operación de la transacción puede hacer que ésta falle (overflow) o errores de lógica.
- Errores locales o condiciones de excepción, por ejemplo saldo insuficiente. Esta podría programarse y no ser un tipo de fallo.



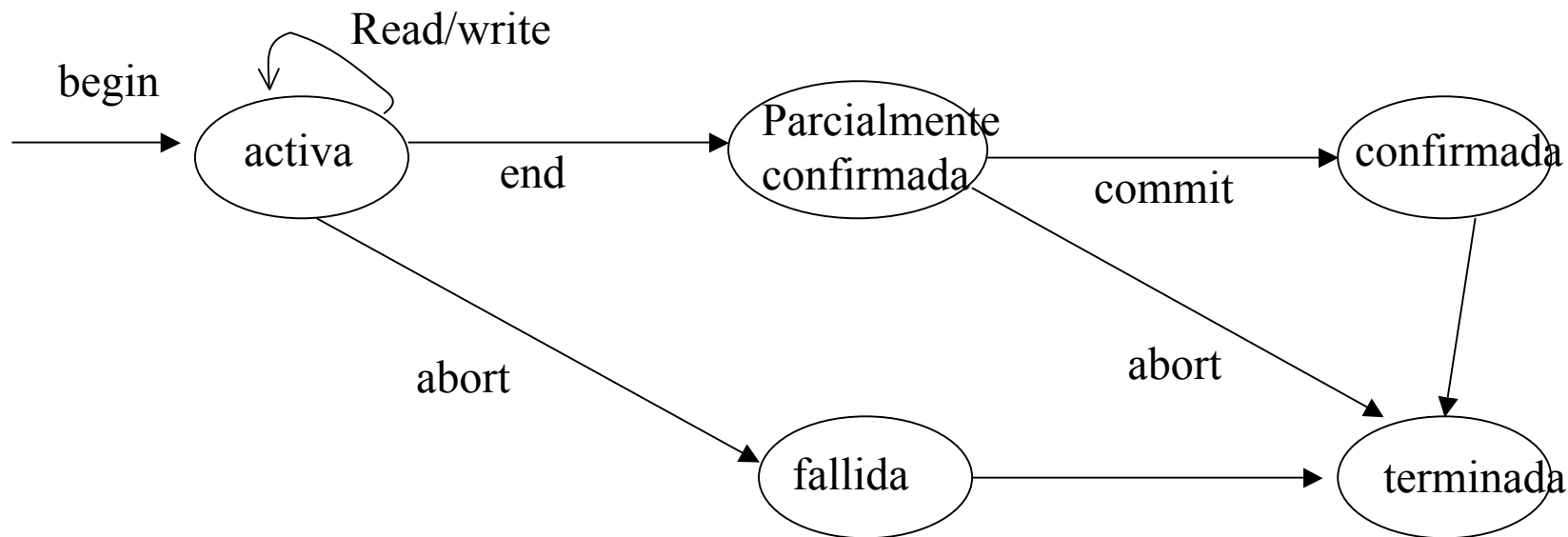
Tipos de Fallos

- Imposición de control de concurrencia: puede suceder que método de control de concurrencia aborte una transacción o deadlock (varias transacciones “pelean” por el mismo recurso)
- Fallo del disco
- Catastrofes



Estado de Transacciones

- Una transacción es una unidad atómica de trabajo que se realiza por completo o bien no se efectúa en absoluto.



Estados de Transacciones



- El componente del sistema encargado de lograr la atomicidad se conoce como administrador de transacciones y las operaciones COMMIT (comprometer) y ROLLBACK (retroceder) son la clave de su funcionamiento.
- La operación COMMIT señala el término exitoso de la transacción: le dice al administrador de transacciones que se ha finalizado con éxito una unidad lógica de trabajo, que la base de datos está o debería estar de nuevo en un estado consistente y que se pueden comprometer, o hacer permanentes todas las modificaciones efectuadas por esa unidad de trabajo



Estados de transacciones

- La operación ROLLBACK, en cambio, señala el término no exitoso de la transacción: le dice al administrador de transacciones que algo salió mal, que la base de datos podría estar en un estado inconsistente y que todas las modificaciones efectuadas hasta el momento por la unidad lógica de trabajo deben retroceder o anularse.

Características de las transacciones



- *Atomicidad*, en el sentido que hemos especificado anteriormente: se ejecutan todas las sentencias o ninguna.
- *Preservación de la consistencia*: la ejecución de una transacción deja la BD en un estado consistente.
- *Aislamiento*, ya que una transacción no muestra los cambios que produce hasta que finaliza.
- *Persistencia*, ya que una vez que finaliza la transacción con éxito, sus efectos perduran en la BD.
- *Seriabilidad*, en el sentido de que el efecto de ejecutar transacciones concurrentemente debe ser el mismo que se produciría al ejecutarlas por separado en un orden secuencial según van entrando en el sistema.



Planes

- Un Plan P de n transacciones: $T_1, T_2, \dots, T_n$ es un ordenamiento para las operaciones, en donde para cada T_i , las operaciones de T_i en P deben aparecer en el mismo orden en que ocurren en T_i .
- El plan P_1 (de slice 9) es:
 - $P_1: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y);$
 - $r = \text{read}, w = \text{write}, a = \text{abort}, c = \text{commit}$



Planes

- Dos operaciones de un plan están en conflicto si satisfacen:
 - Pertenecen a distintas transacciones
 - Tienen acceso al mismo elemento X
 - Al menos una de las operaciones es $\text{write_item}(X)$
 - Para el $P1$, $r1(X)$ y $w2(X)$ están en conflicto



Seriabilidad de planes

- Planes en Serie: las operaciones de cada transacción se ejecutan de manera consecutiva, sin intercalación.
- Los planes en serie limitan la concurrencia, por ejemplo si una transacción está esperando que se complete una operación de E/S, no se puede conmutar el procesador a otra transacción, desperdiciando la CPU.



Seriabilidad de planes

- Un plan de n transacciones es serializable si es equivalente a algún plan en serie.
- Dos planes son equivalentes por conflicto si el orden de dos operaciones cualquiera en conflicto es el mismo en ambos planes



Ejemplo Plan en Serie

● T1	T2
● Read-item(X); ● $X := X - N$; ● Write-item(X); ● Read-item(Y); ● $Y := Y + N$; ● Write-item(Y); PLAN A	Read-item(X); $X := X + M$; write-item(X);



Ejemplo Plan no en Serie

● T1	T2
● Read-item(X); ● $X := X - N$; ● Write-item(X); ● Read-item(Y); ● $Y := Y + N$; ● Write-item(Y);	Read-item(X); $X := X + M$; write-item(X);



Equivalencia de planes

- El plan B es equivalente al plan A. El plan B es serializable
- Para probar si un plan es serializable por conflictos se usa un grafo de precedencia o grafo de serialización, que es un grafo dirigido $G=(N,A)$, con $N = \{T_1, T_2, \dots, T_n\}$ y $A = \{a_1, a_2, \dots, a_n\}$ arcos. Cada arco a_i tiene la forma $(T_j \rightarrow T_k)$, con $j, k = 1, n$, donde T_j es el nodo inicial de a_i y T_k es el nodo final. El arco se crea si una de las operaciones de T_j aparece en el plan antes que alguna operación en conflicto de T_k .



Equivalencia de Planes

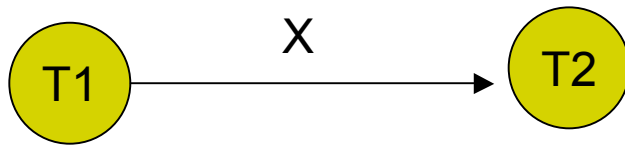
- Algoritmo
- Para cada transacción T_i que participa en el plan crear un nodo etiquetado T_i en el grafo.
- Para cada paso en P en el que T_j ejecuta una orden $r(X)$ después de que T_i ejecute una orden $w(X)$, crear un arco ($T_i \rightarrow T_j$) en el grafo



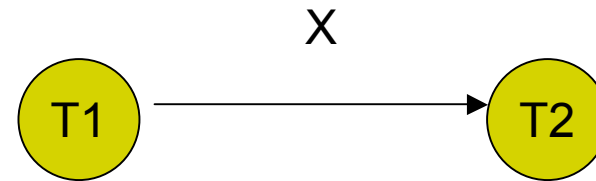
Equivalencia de Planes

- Algoritmo
- Para cada paso en P en que T_j ejecuta una operación $w(X)$ después de que T_i ejecute $r(X)$, crear un arco $(T_i \rightarrow T_j)$ en el grafo
- Para cada paso en P en que T_j ejecuta $w(X)$ después de que T_i ejecute $w(X)$, crear un arco $(T_i \rightarrow T_j)$ en el grafo
- El Plan P es serializable si y solo si el grafo no tiene ciclos.

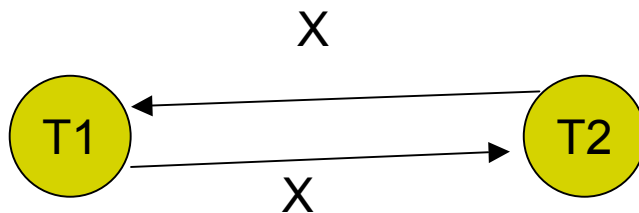
Equivalencia de Planes



Plan A



Plan B



Plan C: No Serializable

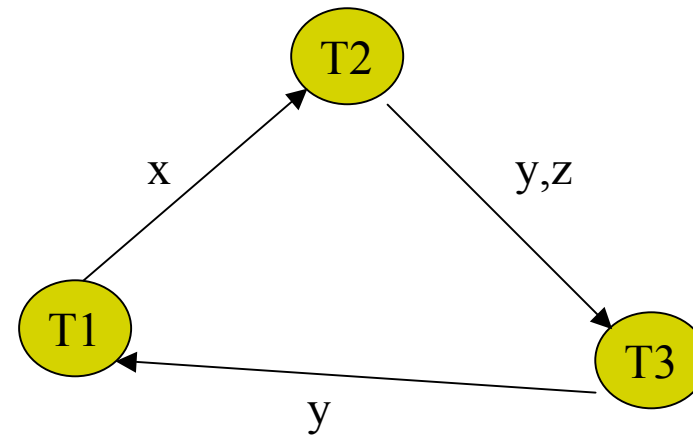
T1	T2
Read-item(X); X:=X-N;	Read-item(X); X:=X+M;
Write-item(X); Read-item(Y);	write-item(X);
Y:=Y+N; Write-item(Y);	

Ejercicio

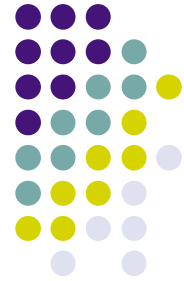
- T1:r(X);w(X);r(Y);w(Y);
- T2:r(Z);r(Y);w(Y);r(X);w(X);
- T3:r(Y);r(Z);w(Y);w(Z);

T1	T2	T3
	r(Z)	
	r(Y)	
	w(Y)	
		r(Y)
		r(Z)
r(X)		
w(X)		
		w(Y)
		w(Z)
	r(X)	
r(Y)		
w(Y)		
	w(X)	

Ver si el Plan es serializable. Construir el grafo de precedencia



No es serializable





Archivo de log

- Para conseguir anular y recuperar transacciones, el método mas usado consiste en utilizar un archivo de diario o log en el que va guardando toda la información necesaria para deshacer (en caso de fracasar) o rehacer (en caso de recuperar) las transacciones. Este archivo consta de:
- identificador de la transacción
- hora de modificación
- identificador del registro afectado
- tipo de acción
- valor anterior del registro
- nuevo valor del registro
- información adicional



Puntos de revisión

- Un concepto relacionado con los archivos de log es el CHECKPOINT, que permite manejar en forma eficiente el contenido de los archivos log, ya que permiten no tener que recorrer todo el archivo de log, ante fallas.
- El establecimiento de puntos de revisión implica:
 - grabar físicamente el contenido de los buffers de datos a la base de datos física
 - grabar físicamente un registro de punto de revisión especial dentro del archivo de log o bitácora

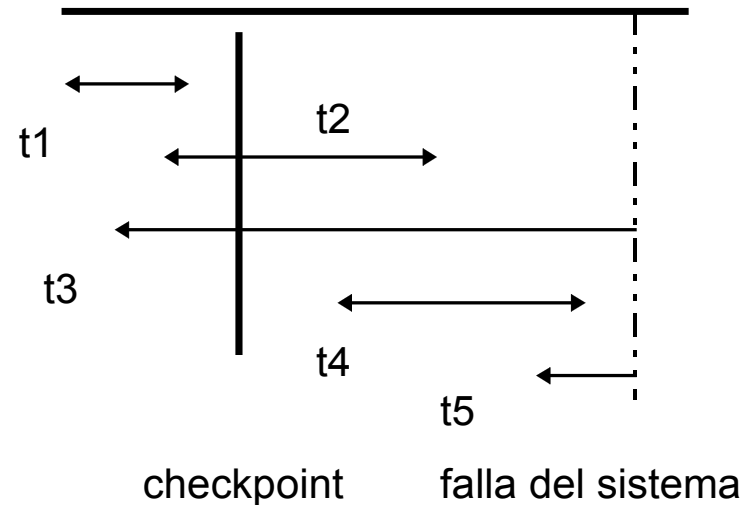


Puntos de revisión

- Los puntos marcados como checkpoint, permiten la recuperación de la base de datos en caliente, es decir, después de la caída del sistema se obtiene la dirección del registro de recuperación más reciente y se recorre el archivo de log desde el punto marcado como checkpoint



Puntos de revisión



La transacción t1 no se ve afectada por la falla del sistema, ni por el proceso de recuperación, por haberse completado antes del último punto de recuperación. Las transacciones t2 y t4, a pesar de haber terminado no han sido grabadas en la base de datos, ya que éstas serían comprometidas en un checkpoint. Las transacciones t3 y t5 deberán rehacerse ya que no han concluido.



Transacciones en SQL

- La definición de transacción en SQL es una unidad lógica de trabajo que garantiza ser atómica.
- No tiene un sentencia de iniciación explícita, pero si terminan con COMMIT o ROLLBACK
- Las característica de las transacciones en SQL se definen con la sentencia SET TRANSACTION, la que especifica modo de acceso, tamaño del área de diagnóstico y nivel de aislamiento.



Modo de acceso

- Puede especificar un READ ONLY o READ WRITE. Un modo READ WRITE permite que se ejecuten operaciones de insertar, modificar, crear o borrar. EN cambio un modo READ ONLY sólo sirve para leer.

Tamaño del área de diagnóstico



- **DIAGNOSTICS SIZE n**, especifica un valor entero que indica el número de condiciones que se pueden tomar simultáneamente en el área de diagnóstico. Estas condiciones suministran información al usuario feedback de errores o excepciones de sentencias SQL ejecutadas más recientemente.



Nivel de aislamiento

- ISOLATION LEVEL <aislamiento>, donde aislamiento puede tomar los sgtes valores:
 - READ UNCOMMITTED (lectura no confirmada)
 - READ COMMITTED (lectura confirmada)
 - REPEATABLE READ (lectura repetible)
 - SERIALIZABLE
- Si se usa algo menos restrictivo que serializable, pueden ocurrir una de las siguientes violaciones.



Violaciones (1)

- Lectura sucia: Ocurre cuando una transacción usa los valores actualizados por otra transacción aún no confirmada y que aborta.
- Lectura no repetible: Se leen dos veces un mismo valor que no es repetible.
- Fantasmas: Después de leer datos, se crean nuevos que aparecen cuando se realiza una nueva lectura.



Violaciones (2)

Nivel de Aislamiento	Lectura sucia	Lectura no repetible	Fantasmas
Lectura no confirmada	Si	Si	Si
Lectura confirmada	No	Si	Si
Lectura repetible	No	No	Si
Serializable	No	No	No



Ejemplo de SQL

```
EXEC SQL WHENEVER SQLERROR GOTO UNDO;  
EXEC SQL SET TRANSACTION  
    READ WRITE  
    DIAGNOSTICS SIZE 5  
    ISOLATION LEVEL SERIALIZABLE;  
EXEC INSERT...;  
EXEC UPDATE..;  
EXEC COMMIT;  
GOTO: THE_END;  
UNDO: EXEC SQL ROLLBACK;  
THE_END:....;
```

Técnicas de Control de Concurrency



- Pesimistas: bloqueo y marcas de tiempo
- Optimistas
- **Técnicas de bloqueo:** es una variable asociada a cada elemento de datos que describe el estado de dicho elemento respecto a las posibles operaciones (recuperación o actualización) que se pueden realizar sobre ellos en cada momento.



Técnicas de bloqueos

- Las transacciones pueden llevar a cabo bloqueos, impidiendo a otros usuarios la recuperación o actualización de los elementos bloqueados, para evitar inconsistencias en el acceso concurrente.
- Los SGBD tienen bloqueos (por tupla, por tabla) para asegurar la consistencia. Los usuarios también pueden bloquear explícitamente los objetos, impidiendo el acceso por parte de otros usuarios.



Técnicas de bloqueos

- **Tipos de bloqueo**
- Exclusivos o binario: cuando una transacción mantiene un bloqueo de este tipo, ninguna otra transacción puede acceder al objeto bloqueado, ni bloquearlo, hasta que sea liberado por la transacción que lo había retenido. Se utiliza cuando se quiere actualizar datos.
- Bloqueo compartido: cuando una transacción bloquea en este modo, permite que otras transacciones retengan también el objeto en bloque compartido, pero no exclusivo. Este tipo se utiliza cuando no se requiere actualizar datos, pero se desea impedir cualquier modificación mientras los datos son consultados.

Reglas de esquema de bloqueos (1)



- Una transacción T debe emitir un bloqueo de lectura sobre X o bloqueo de escritura sobre X antes de que realice cualquier operación de lectura de X de T.
- Una transacción T debe emitir un bloqueo de escritura sobre X antes de que realice cualquier operación de escritura de X de T.
- Una T debe emitir la operación desbloquear(X) una vez que se hayan completado las operaciones de leer o escribir X.

Reglas de esquema de bloqueos (2)



- Una transacción no emitirá una operación de bloqueo de lectura sobre X si ya posee un bloqueo de lectura compartido o de escritura sobre el elemento X . Esto tiene algunas excepciones.
- Una transacción no emitirá una operación de bloqueo de escritura sobre X si ya posee un bloqueo de lectura compartido o de escritura sobre el elemento X .
- Una transacción T no emitirá una operación de desbloquear X a menos que ya posea un bloqueo de lectura compartido o escritura para X

Reglas de esquema de bloqueos (3)



- En algunas ocasiones es posible la conversión de bloqueos, lo que permite que un bloqueo de lectura pase a escritura o vice versa. Para ello, la tabla de bloqueos debe incluir identificación de transacciones en el registro de bloqueo y así saber qué transacciones poseen bloqueos.



Bloqueo de 2 Fases

- El bloqueo por si sólo no garantiza la seriabilidad porque desbloques pueden ocurrir antes de tiempo.
- Para garantizar la seriabilidad se usa un protocolo denominado bloqueo de dos fases.



Bloqueo de 2 Fases

- Se dice que una transacción sigue el protocolo de dos fases, si TODAS las operaciones de bloqueo preceden la primera operación de desbloqueo. Entonces tenemos una fase de expansión y otra de contracción
- Si se permite la conversión en bloqueos, los cambios de promoción (lectura a escritura) ocurren en la expansión y los de degradación en la contracción.
- El bloqueo de dos fases puede limitar la concurrencia.

Tipos de bloqueos de dos fases (1)



- El bloqueo descrito anteriormente es el **básico**.
- El bloqueo de dos fases **conservador** o estático requiere que se bloqueen todos los elementos a los que tendrá acceso antes de comenzar a ejecutarse.
- El bloqueo **estricto** es en la práctica el más usado.



Bloqueo de 2 Fases estricto

- Cada transacción debe obtener un bloqueo compartido antes de leer X y un bloqueo exclusivo antes de escribir X
- Una transacción no libera ningún bloqueo exclusivo antes de confirmar o abortar.
- Si una T mantiene un bloqueo exclusivo sobre un objeto O , ninguna otra transacción puede tener un bloqueo compartido o exclusivo sobre O .
- Una transacción no puede pedir bloqueos adicionales sobre O una vez que libera un bloqueo de O .
- Bloqueo de 2 Fases estricto permite sólo planes cuyo grafo de precedencia es acíclico.



Tipos de bloqueos de 2 fases

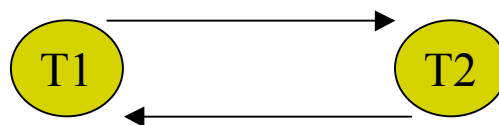
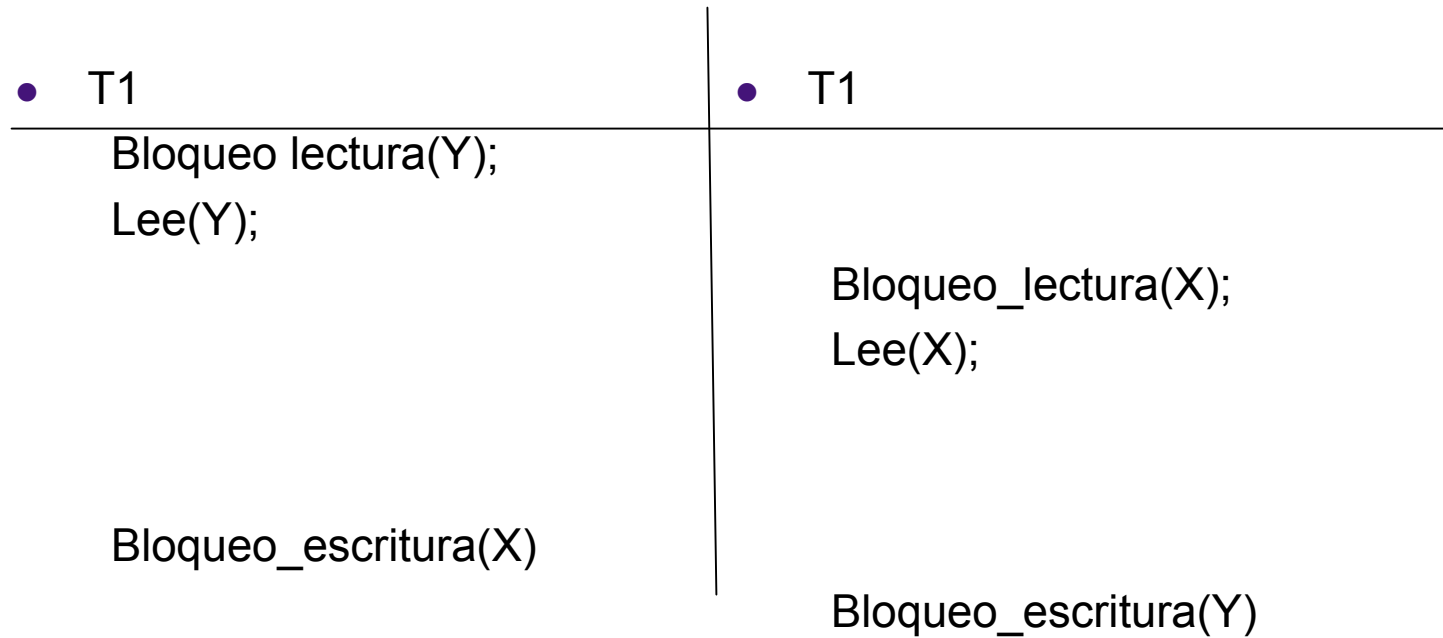
- Una variación del bloqueo de dos fases estricto es el **riguroso**, en el cual ningún tipo de bloqueo es liberado antes de confirmar o abortar una transacción.



Técnicas de bloqueo

- El problema de las técnicas de bloqueo es que puede producirse un interbloqueo (deadlock) o bloqueo mortal, dos o más transacciones están esperando cada una de ellas que la otra libere algún objeto antes de seguir.

Bloqueo Mortal



Grafo de espera

Protocolo de prevención de deadlock (1)



- **Bloqueo por adelantado:** obliga a que las transacciones bloqueen todos los elementos que necesitan por adelantado. En caso de no poder conseguir todos esos elementos no bloquea ninguno y se queda en espera hasta volver a intentarlo.
- **Ordenación:** Ordena todos los elementos de la base de datos y una transacción que necesite varios de esos elementos los bloqueará en ese orden. Esto no es muy práctico.

Protocolo de prevención de deadlock (2)



- **Marca de tiempo transaccional:** Las transacción están marcadas con el tiempo de de iniciación de ellas. Si T_1 se inicia antes que T_2 , entonces $MT(T_1) < MT(T_2)$. Bajo este protocolo hay dos esquemas que evitan el bloqueo mortal.
- Supongamos que T_i intenta bloquear un elemento X que ya está bloqueado por T_j con un bloqueo en conflicto, entonces las reglas que se aplican son:
 - Esperar_morir: Si $MT(T_i) < MT(T_j)$, entonces T_i puede esperar, en caso contrario T_i se aborta y se reinicia con la misma marca de tiempo.
 - Herir_esperar: Si $MT(T_i) < MT(T_j)$, se aborta T_j y reinicia posteriormente con la misma marca de tiempo. EN caso contrario, T_i puede esperar.

Protocolo de prevención de deadlock (3)



- **No-espera:** Si una transacción no puede bloquear se aborta y reinicia posteriormente.
- **Espera-cautelosa:** Si T_j no está detenida y tiene bloqueado un elemento que T_i quiere bloquear, entonces T_i se detiene, de lo contrario se aborta T_i .



Detección de Deadlock

- Se controla de forma periódica si se ha producido un deadlock. Se construye un grafo en espera, cada nodo es una transacción en ejecución y un arco de una transacción T_i a T_j , en caso que T_i esté esperando un elemento que ocupa T_j . Si existe un ciclo en el grafo tenemos un deadlock. La solución es escoger transacciones víctimas y deshacerlas, hasta que desaparezca el deadlock. Cada SGBD tiene políticas diferentes para escoger víctimas.
- Otro esquema de detección es tener un tiempo limitado de espera de una transacción, el cual al ser excedido, implica que ha ocurrido un deadlock.



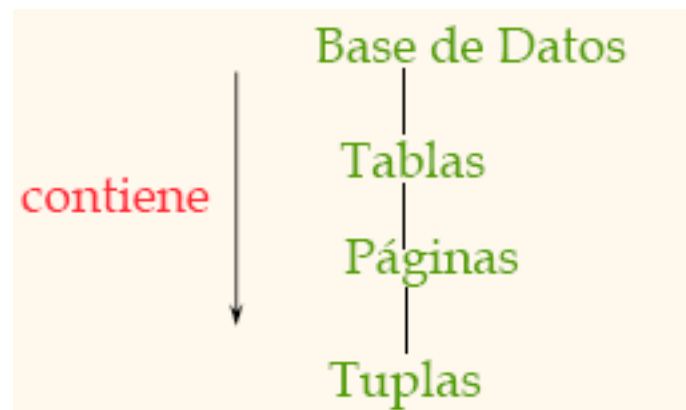
Técnicas de bloqueo

- Este tema influye notoriamente en el rendimiento de los sistemas. Los SGBD pueden bloquear:
 - un campo de un registro (un atributo de una tabla)
 - un registro (una tupla)
 - un archivo (una tabla)
 - la BD total
- Esto se llama *granularidad* del bloqueo.



Técnicas de bloqueo

- *Granularidad* muy gruesa implica gestionar menor nro de bloqueos, pero retrasa la ejecución de muchas transacciones (los objetos no se van liberando). Una granularidad muy fina, permite mayor concurrencia, pero aparecen mas situaciones de deadlock que han de ser resueltas.



Modos de Bloqueo: Granularidad



- ❖ Permitir a las transacciones bloquear a cada nivel, pero con un protocolo especial usando las nuevas “**intenciones**” de bloqueo:
- ❖ Antes de bloquear un item, T debe declarar “**intención**” de bloqueo en todos sus ancestros.
- ❖ Para desbloquear, vaya de lo específico a lo general
- ❖ **Modo SIX:** Como S & IX al mismo tiempo.

	--	IS	IX	S	X
--	✓	✓	✓	✓	✓
IS	✓	✓	✓	✓	
IX	✓	✓	✓		
S	✓	✓		✓	
X	✓				



Protocolo de bloqueo múltiple

- ❖ Cada transacción parte de la raíz de la jerarquía.
- ❖ Para obtener un lock S o IS sobre un nodo, debe mantener IS o IX sobre el nodo padre.
 - ¿Que si T mantiene SIX (o S) sobre el padre?
- ❖ Para obtener X o IX o SIX sobre un nodo, debe mantener IX o SIX sobre el nodo padre.
- ❖ Debe liberar locks de abajo a arriba.

El protocolo es correcto en el sentido que es equivalente a poner locks en los niveles de hojas de la jerarquía.

Ejemplo



- ❖ T1 barre R, y actualiza unas cuantas tuplas:
 - T1 obtiene un lock SIX sobre R, y luego repetidamente obtiene locks S sobre las tuplas de R, y a veces se promociona a X sobre las tuplas.
- ❖ T2 usa un índice para leer sólo parte de R:
 - T2 obtiene un lock IS sobre R, y luego obtiene locks S sobre tuplas de R.
- ❖ T3 lee todo R:
 - T3 obtiene un lock S sobre R.
 - o, T3 puede comportarse como T2; y puede decidir cuál lee y lockear S.

	--	IS	IX	S	X
--	✓	✓	✓	✓	✓
IS	✓	✓	✓	✓	
IX	✓	✓	✓		
S	✓	✓		✓	
X	✓				

Técnicas de marca de tiempo



- **Técnicas de marca de tiempo (timestamping)**
- Las marcas de tiempo son identificadores únicos que se asignan a las transacciones, que se consideran como el tiempo de inicio de una transacción. Con esta técnica no existen bloqueos. Ordena las transacciones. Se retrasan.



Técnicas Optimistas

- Las transacciones acceden libremente a los elementos, y antes de finalizar se determina si ha habido interferencias
- Este tipo de técnicas considera que las transacciones tienen 3 fases:
- *Lectura*: las transacciones realizan operaciones sobre copias privadas de los objetos (accesibles solo por la transacción)
- *Validación* : en la que se comprueba si el conjunto de objetos modificados por una transacción se solapa con el conjunto de objetos modificados por alguna otra que haya hecho la validación durante la fase de lectura de dicha transacción
- *Grabación*:, en el caso de no detectar interferencias se graban las modificaciones, convirtiendo las versiones privadas de los objetos en versiones actuales



Integridad

- La integridad tiene como función proteger la BD contra operaciones que introduzcan inconsistencias en los datos. Se habla de integridad en el sentido de corrección, validez o precisión de los datos.
- El subsistema de integridad de un SGBD debe por tanto detectar y corregir, en la medida de lo posible, las operaciones incorrectas. En la práctica es el punto débil de los SGBD comerciales, ya que casi toda la verificación de integridad se realiza mediante código de procedimientos escritos por los usuarios.
- Habrá operaciones cuya falta de corrección no sea detectable, por ejemplo, introducir un fecha de nacimiento 25/12/1945 cuando en realidad era 25/12/1954

Operaciones semánticamente inconsistentes



- Son las que transgreden las restricciones que ha definido el administrador al diseñar la base de datos, tales como:
 - Restricciones sobre dominios por ejemplo, el dominio edad esté comprendido entre 18 y 65 años.
 - Restricciones sobre los atributos, por ejemplo, la edad de los empleados ingenieros debe ser mayor de 21 años.
- Estas restricciones pueden ser estáticas, como las anteriores o dinámicas por ejemplo, el sueldo de un empleado no puede disminuir.

Operaciones semánticamente inconsistentes



- Otra forma de clasificar las restricciones es en:
- simples: si se aplican a una ocurrencia de un atributo con independencia de los demás, por ejemplo, el sueldo de un empleado tiene que ser mayor que 60000.
- compuestas: si implican más de una ocurrencia, como es el caso de las restricciones de comparación, por ejemplo, el sueldo de un empleado debe ser menor que el de su jefe. O bien, las llamadas de globalidad, por ejemplo, el sueldo medio de los empleados de un determinado depto debe ser menor de 250000



Reglas de Integridad

- En general, una regla de integridad está compuesta por tres componentes:
- La restricción propiamente tal, que establece la condición que deben cumplir los datos
- La respuesta a la transgresión, que especifica las acciones a tomar, como rechazar las operaciones, informar al usuario, corregir el error con acciones complementarias, etc.
- Condición de disparo, que especifica cuándo debe desencadenarse la acción especificada en la restricción de integridad: antes, después o durante cierto evento



Seguridad

- El objetivo es proteger la BD contra accesos no autorizados. Se llama también privacidad.
- Incluye aspectos de:
 - Aspectos legales, sociales y éticos
 - Políticas de la empresa, niveles de información pública y privada
 - Controles de tipo físico, acceso a las instalaciones
 - Identificación de usuarios: voz, retina del ojo, etc.
 - Controles de sistema operativo

Seguridad



- En relación al SGBD, debe mantener información de los usuarios, su tipo y los accesos y operaciones permitidas a éstos.
- Tipos de usuarios:
- DBA, están permitidas todas las operaciones, conceder privilegios y establecer usuarios
- Usuario con derecho a crear, borrar y modificar objetos y que además puede conceder privilegios a -otros usuarios sobre los objetos que ha creado.
- Usuario con derecho a consultar, o actualizar, y sin derecho a crear o borrar objetos



Seguridad

- Privilegios sobre los objetos, añadir nuevos campos, indexar, alterar la estructura de los objetos, etc. Los SGBD tienen opciones que permiten manejar la seguridad, tal como GRANT, REVOKE, etc. También tienen un archivo de auditoría en donde se registran las operaciones que realizan los usuarios.
- Otro mecanismo de seguridad que ofrecen los SGBD es entregar información a los usuarios a través de vistas (CREATE VIEW)